

Library Management System

Softwareentwicklung 2, WiSe22/23

<https://gitlab.mi.hdm-stuttgart.de/joo41/LMS.git>

—

□

×

New User

New Book

Edit Book

Issue Book

Return Book

Create

Show

isbn='3836283921', title='Java: Der Grundkurs', author='Michael Kofler', issued='No'

isbn='3969672244', title='Python für Einsteiger', author='Florian Dalwigk', issued='No'

Title

Author

ISBN

Shaqaiq, Adib as448@hdm-stuttgart.de

Öhler, Jonas joo41@hdm-stuttgart.de

02.02.2023

1. Entwurfsbeschreibung

Bei unserem Projekt handelt es sich um ein "Library Management Program".

Das Programm ermöglicht Bibliotheken, ihre Bücher zu verleihen, ausgeliehene Bücher zurückzugeben und Bücher zu verwalten.

Es können neue Benutzer erstellt und neue Bücher angelegt werden. Des Weiteren besteht die Möglichkeit, Bücher zu bearbeiten und zu löschen. Die erstellten Bücher können dann an Benutzer verliehen oder zurückgegeben werden.

Die ausgeliehenen Bücher werden der Liste eines Users hinzugefügt. Die verliehenen Bücher erhalten eine Markierung, sodass sie nicht an weitere Benutzer verliehen werden können.

Um ein Buch zurückzugeben, muss der entsprechende Benutzer ausgewählt und aus der Liste seiner ausgeliehenen Bücher das entsprechende Buch markiert werden.

Dem Benutzer des Programms ist es möglich, die Bücher hinsichtlich Name, Autor und ISBN zu verändern. Es wird außerdem darauf geachtet, dass bereits ausgeliehene Bücher mit verändert werden.

2. Dokumentation des Projekts

Architektur

Interface: Unser Interface ist eine Erweiterung von Serializable. Serialisierung wird benötigt, um Objekte in Byte Stream zu konvertieren. Unser Interface erweitert es um zwei Methoden, um den Bitstream zu speichern und zu lesen, damit Objekte aus Json-Files generiert und in diese gespeichert werden können.

Enum: Zum einen nutzen wir Enums, um den Status unserer Bücher wiederzugeben. Zum anderen, um zu unterscheiden, ob ein User bzw. ein Buch in einer Json-File gespeichert werden soll.

Vererbung: Unsere Klassen "BookManager" und "UserManager" implementieren das Interface.

Ordnerstruktur

Unser LMSPackage bildet unser Haupt Package. Darin befinden sich vier weitere Packages inklusive der Main Klasse.

BookPackage: Beinhaltet alle Klassen die zu Book gehören.

GuiPackage: Beinhaltet alle Controller.

HelperPackage: Stellt die Verbindung zur Datenbank her und steuert unsere threads.

userPackage: beinhaltet alle Klassen, die zu User gehören.

Clean Code

In keiner Klasse wurden public members verwendet.

Wir haben darauf geachtet, nur wenige static methods zu implementieren. Es gibt keine getter, die es ermöglichen, den Wert eines private members zu verändern.

GUI

In unserer GUI haben wir mehrere action events implementiert, die beispielsweise auf einen Buttonclick reagieren. Des Weiteren wurden mehrere Screens eingebaut, die durch eine load scene function gesteuert werden.

Logging

log.error: immer dann, wenn Szenen nicht geladen werden können oder nichts ausgewählt wurde.

log.debug: um den Zwischenstand unserer Methoden anzeigen zu lassen.

log.info: um anzuzeigen, dass eine Methode erfolgreich abgeschlossen wurde.

UML

Wir haben mit dem Use Case Diagram begonnen, um eine grobe Vorstellung zu erhalten, welche Klassen wir implementieren wollen. Daraus entwickelten wir dann das Klassendiagramm, indem wir geeignete Methoden und Attribute zu den einzelnen Klassen hinzufügen. Zusätzlich haben wir noch ein Sequenzdiagramm erstellt.

Threads

Lesen oder schreiben Json - Files. Der lese-thread wird in der main ausgeführt und läuft bis das Programm geschlossen wird. der schreib-thread wird immer funktionsweise aufgerufen.

Streams und Lambda-Funktionen

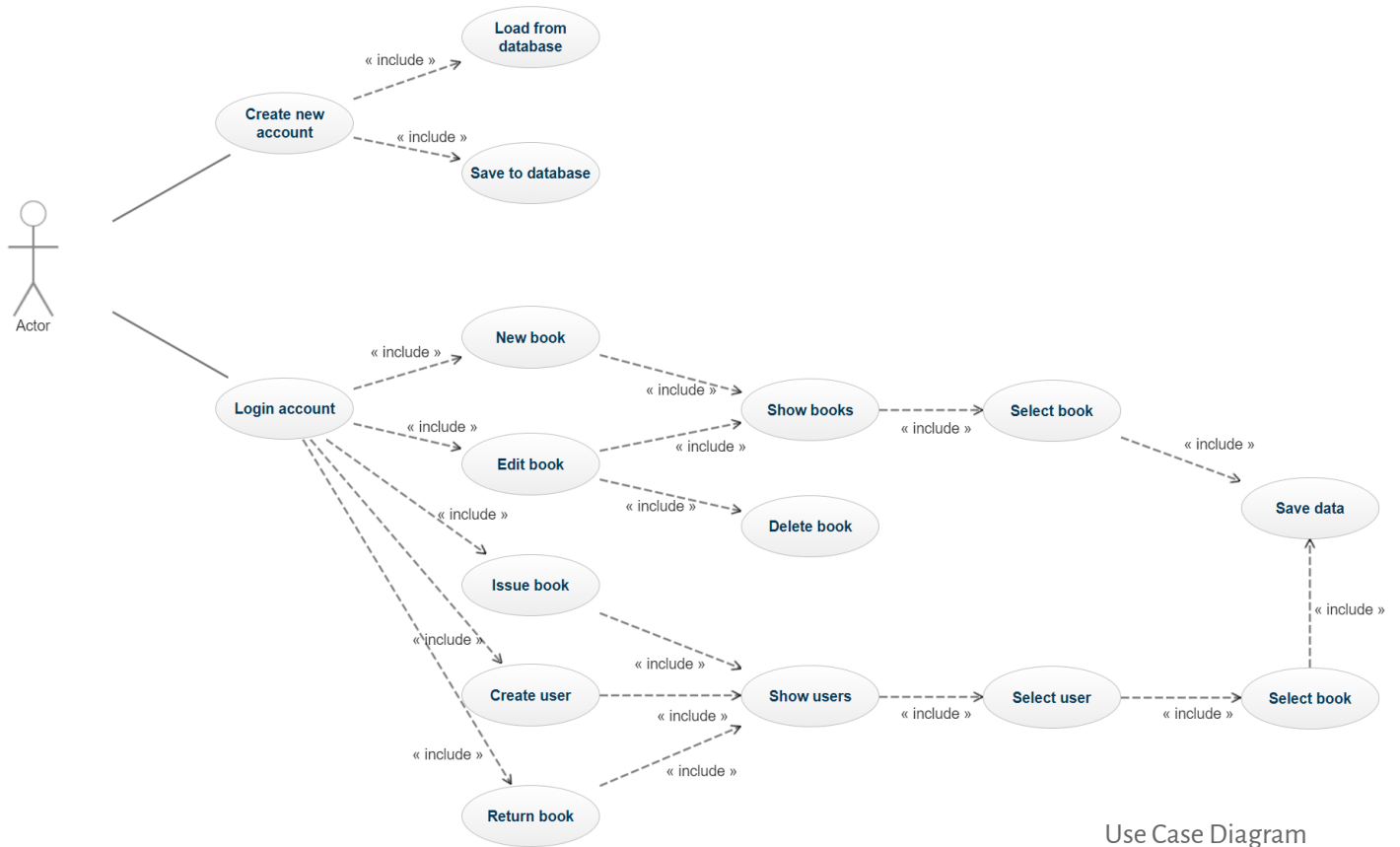
Unsere Streams durchsuchen anhand der übergebenen isbn oder id, die entsprechende ArrayList.

Falls etwas gefunden wird, wird der entsprechende User bzw. das entsprechende Buch zurückgegeben.

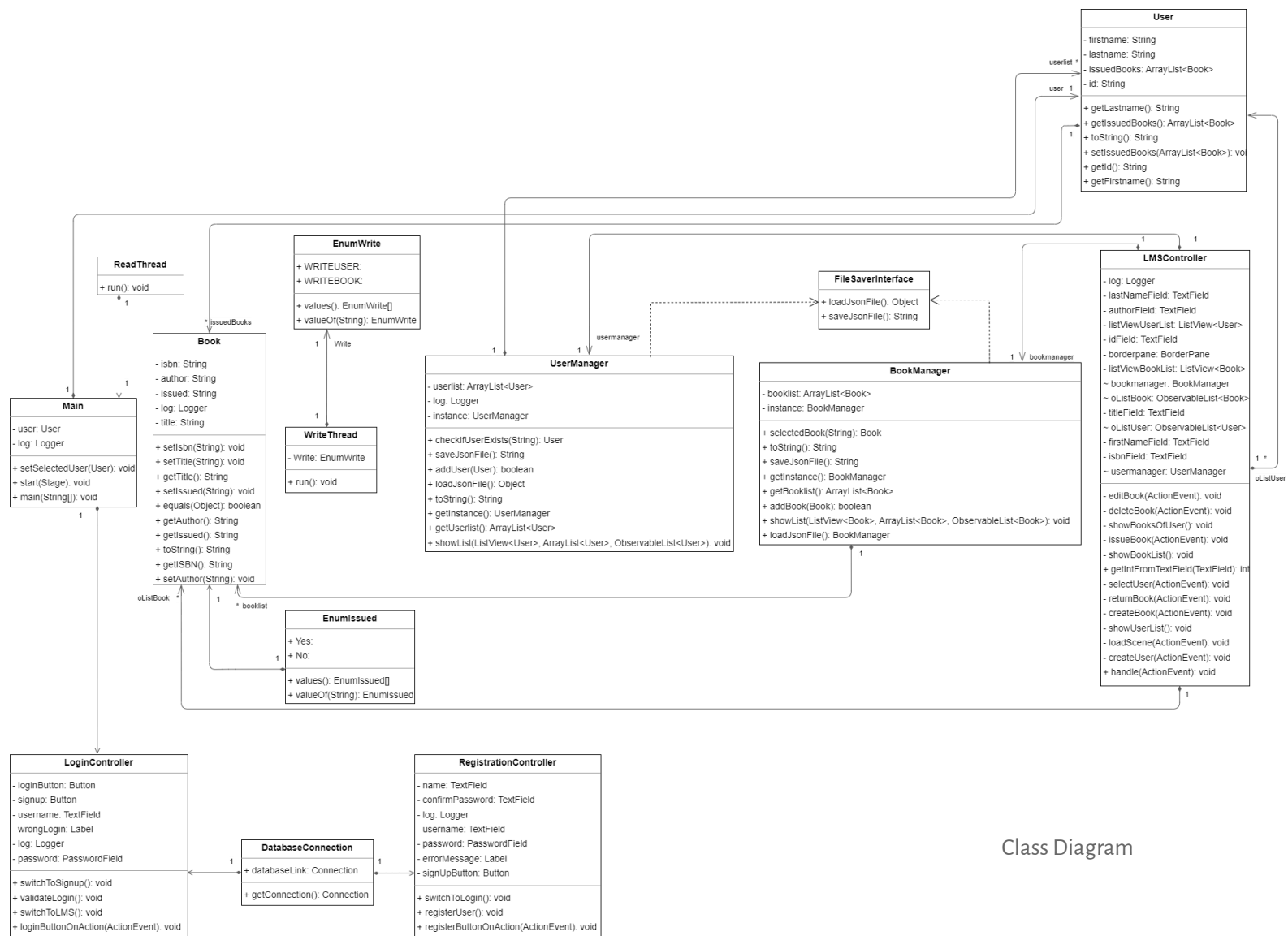
3. Bewertungsbogen

Vorname	Nachname	Kürzel	Matrikelnummer	Projekt	Arc.	Clean Code	Doku	Tests	GUI	Logging/Except.	UML	hhh	Streams	Profiling	Summe - Projekt	Kommentar	Projekt-Note
Jonas	Öhler	jo041	44338	LMS	2	2	3	0	3	3	3	3	3	3	25,00		1,70
Adib	Shaqaiq	as448	44163	LMS	2	2	3	0	3	3	3	3	3	3	25,00		1,70

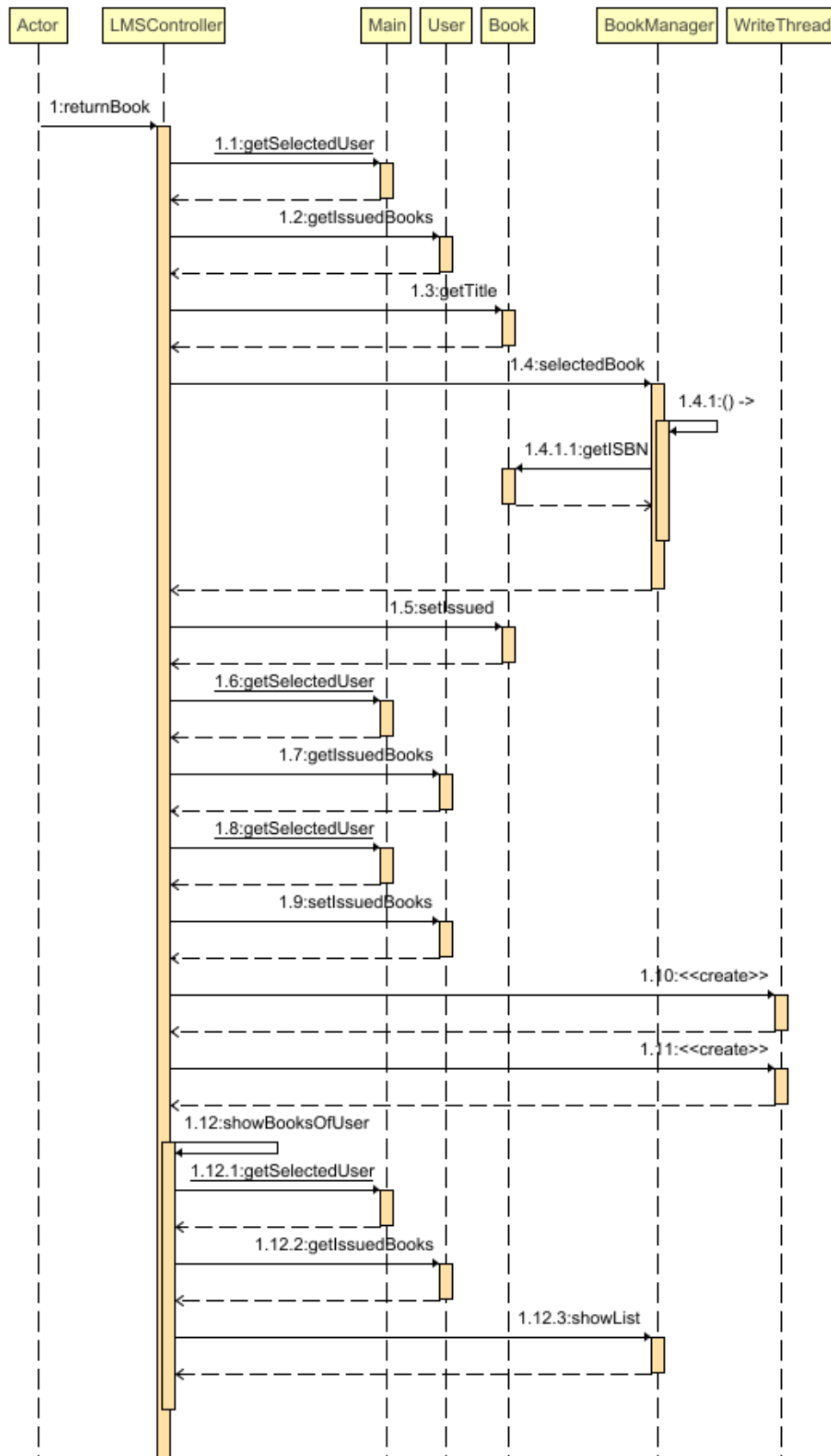
4. UML



Use Case Diagram



Class Diagram



Sequence Diagram